

4.1 LE TEMPLATE = UN FORMULAIRE PRÉ-IMPRIMÉ

Imaginez un formulaire administratif avec des cases vides.
Django remplit les cases avec les données réelles au moment de l'affichage.

EXEMPLE :

```
<!-- template HTML -->
<h1>Bienvenue, {{ user.username }} !</h1>
<p>Votre solde : {{ compte.solde }} FCFA</p>
```

Au moment de l'affichage, Django remplace :

- `{{ user.username }}` → "Isac"
- `{{ compte.solde }}` → "45 000"

Résultat visible dans le navigateur :

```
"Bienvenue, Isac !"
"Votre solde : 45 000 FCFA"
```

4.2 SYNTAXE DE BASE DES TEMPLATES DJANGO

SYNTAXE	USAGE
<code>{{ variable }}</code>	Afficher une valeur
<code>{% if condition %}</code>	Affichage conditionnel
<code>{% for x in liste %}</code>	Boucle sur une liste de données
<code>{% block nom %}</code>	Zone remplaçable (héritage de template)
<code>{% extends "base" %}</code>	Hériter d'un template parent
<code>{% include "frag" %}</code>	Inclure un sous-template
<code>{% url 'nom_vue' %}</code>	Générer une URL depuis son nom
<code>{% csrf_token %}</code>	Sécurité formulaire (OBLIGATOIRE dans POST)
<code>{{ val upper }}</code>	Filtres : upper, lower, date, floatformat...

4.3 PERSONNALISER LE NOM ET LES COULEURS – MINIMUM VITAL

FICHIER À MODIFIER EN PRIORITÉ : `templates/base.html`

Cherchez et remplacez :

- "Boutique ERP" → Nom de votre organisation
- "Gykhamine" → Votre marque (ou laisser tel quel)
- Logo : remplacez `static/img/logo.png` par votre logo (même nom de fichier)

FICHIER CSS : `static/css/style.css` (ou `main.css` selon la capsule)

```
:root {
  --couleur-principale : #0d6efd; ← Changer cette valeur hex
  --couleur-menu       : #1a1a2e; ← Couleur du menu latéral
  --couleur-fond       : #f8f9fa; ← Couleur de fond des pages
}
```

Pour trouver un code couleur hex : cherchez "color picker" dans Google.

4.4 ADAPTER UN TABLEAU DE DONNÉES – CAS CONCRET

TEMPLATE ORIGINAL (boutique) :

```
<table>
  <tr><th>Produit</th><th>Prix</th><th>Stock</th></tr>
  {% for p in produits %}
  <tr>
    <td>{{ p.nom }}</td>
    <td>{{ p.prix_vente }} FCFA</td>
```

```

        <td>{{ p.stock }}</td>
    </tr>
    {% endfor %}
</table>

```

ADAPTÉ POUR UNE GESTION DE PATIENTS (clinique) :

```

<table>
  <tr><th>Patient</th><th>Diagnostic</th><th>Chambre</th><th>Statut</th></tr>
  {% for patient in patients %}
  <tr class="{% if patient.statut == 'urgent' %}table-danger{% endif %}">
    <td>{{ patient.nom_complet }}</td>
    <td>{{ patient.diagnostic }}</td>
    <td>{{ patient.chambre.numero|default:"Non assigné" }}</td>
    <td>
      {% if patient.statut == 'urgent' %}
      <span style="color:red;">⚠ URGENT</span>
      {% else %}
      {{ patient.statut|capfirst }}
      {% endif %}
    </td>
  </tr>
  {% endfor %}
</table>

```

CHANGEMENTS EFFECTUÉS : noms des colonnes, champs affichés, logique de coloration conditionnelle. La structure <table>/<tr>/<td> reste identique.

4.5 LES FILTRES DJANGO LES PLUS UTILES POUR L’AFFICHAGE

FILTRE	EXEMPLE DE RÉSULTAT
{{ val floatformat:0 }}	1250.75 → "1251"
{{ val intcomma }}	1250000 → "1,250,000"
{{ date date:"d/m/Y" }}	2025-01-15 → "15/01/2025"
{{ texte truncatewords:15 }}	Texte tronqué à 15 mots...
{{ val default:"Non défini" }}	Affiche "Non défini" si vide
{{ val yesno:"Oui,Non" }}	True → "Oui", False → "Non"
{{ val upper }}	"nom" → "NOM"
{{ val capfirst }}	"jean" → "Jean"

RÉSUMÉ DU CHAPITRE 4

- ✓ {{ variable }} = cases vides remplies par Django
- ✓ {% for %} / {% if %} = logique d’affichage sans Python visible
- ✓ Personnalisation minimale : nom, logo, couleurs CSS (3 variables)
- ✓ Adapter un tableau = changer colonnes et champs, pas la structure
- ✓ Les filtres Django formatent les données sans toucher aux vues

→ SUITE : Chapitre 5 – Analyse de données intégrée (Data Science en contexte)